84. Символьные константы строк C-style

Мы уже знаем, как создать и инициализировать строку C-style:

```
#include <iostream>

int main()
{
    char myName[] = "John";
    std::cout << myName;

    return 0;
}
```

Язык C++ поддерживает еще один способ создания символьных констант строк C-style — через **указатели**:

```
#include <iostream>

int main()
{
    const char *myName = "John";
    std::cout << myName;

    return 0;
}
```

Хотя обе эти программы работают и выдают одинаковые результаты, выделение памяти в них выполняется по-разному.

В первом случае в программе выделяется память для **фиксированного массива** длиной 5 и инициализируется эта память строкой `John\0`. Поскольку память была специально выделена для массива, то мы можем изменять её содержимое. Сам массив рассматривается как обычная локальная переменная, поэтому, когда он выходит из **области видимости**, память, используемая им, освобождается для других объектов.

Что происходит в случае с **символьной константой**? Компилятор помещает строку `John\0` в память типа read-only (только чтение), а затем создает указатель, который указывает на эту строку. Несколько строковых литералов с одним и тем же содержимым могут указывать на один и тот же адрес. Поскольку эта память доступна только для чтения, а также потому, что внесение изменений в строковый литерал может повлиять на дальнейшее его использование, лучше всего перестраховаться, объявив строку константой (типа const). Также, поскольку строки, объявленные таким образом, существуют на протяжении всей жизни программы (они имеют статическую продолжительность, а не автоматическую, как большинство других локально определенных литералов), нам не нужно беспокоиться о проблемах, связанных с областью видимости. Поэтому следующее в порядке вещей:

```
const char* getName()
{
    return "John";
}
```

В фрагменте, приведенном выше, функция getName() возвращает указатель на строку C-style `John`. Всё хорошо, так как `John` не выходит из области видимости, когда getName() завершает свое выполнение, поэтому вызывающий объект всё равно имеет доступ к строке.

std::cout и указатели типа char

На этом этапе вы, возможно, уже успели заметить то, как std::cout обрабатывает указатели разных типов. Рассмотрим следующий пример:

```
#include <iostream>

int main()
{
    int nArray[5] = { 9, 7, 5, 3, 1 };
    char cArray[] = "Hello!";
    const char *name = "John";

    std::cout << nArray << '\n'; // nArray распадается в указатель типа int
    std::cout << cArray << '\n'; // cArray распадается в указатель типа char
    std::cout << name << '\n'; // name уже и так является указателем типа char

    return 0;
}
```

Результат выполнения программы на моем компьютере:

```
0046FAE8
Hello!
John
```

Почему в массиве типа int выводится адрес, а в массивах **типа char** — строки?

Дело в том, что при передаче указателя не типа char, в результате выводится просто содержимое этого указателя (адрес памяти). Однако, если вы передадите объект типа char* или const char*, то std::cout предположит, что вы намереваетесь вывести строку. Следовательно, вместо вывода значения указателя — выведется строка, на которую тот указывает!

Хотя это всё замечательно в 99% случаев, но это может привести и к неожиданным результатам, например:

```
#include <iostream>

int main()
{
    char a = 'R';
    std::cout << &a;

    return 0;
}
```

Здесь мы намереваемся вывести адрес переменной `a`. Тем не менее, `&a` имеет тип `char*`, поэтому std::cout выведет это как строку!

Результат выполнения программы на компьютере:

```
R╠╠╠╠╠╝╕4;¿■A
```

Почему так? std::cout предположил, что `&a` (типа `char*`) является строкой. Поэтому сначала вывелось `R`, а затем вывод продолжился. Следующим в памяти был мусор. В конце концов, std::cout столкнулся с ячейкой памяти, имеющей значение `0`, которое он интерпретировал как нуль-терминатор, и, соответственно, прекратил вывод. То, что вы видите в результате, может отличаться, в зависимости от того, что находится в памяти после переменной `a`.

Подобное вряд ли случится с вами на практике (так как вы вряд ли захотите выводить адреса памяти), но это хорошая демонстрация того, как всё работает «под капотом» и как программы могут *случайно* «сойти с рельсов».